



Regression Models

Lecture 4

肖升生 博士

Dr. Shengsheng Xiao

Email: Xiao.shengsheng@shufe.edu.cn



Outline

- Classification and Regression
- Linear Regression
 - OLS/MLE /MAP Estimator
 - Overfitting issue
- Ridge Regression
- Lasso Regression



Outline

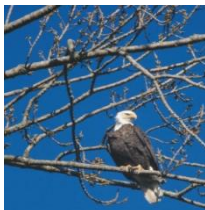
- Classification and Regression
- Linear Regression
 - OLS/MLE /MAP Estimator
 - Overfitting issue
- Ridge Regression
- Lasso Regression

Prediction Problem

- Input: $\mathbf{x} = (x_1, x_2, \dots, x_d)$
- Predict the output (class label) $y \in \mathcal{Y}$
 - Binary classification: $\mathcal{Y} \in \{0, 1\}$
 - Multi-class classification: $\mathcal{Y} = \{1, 2, \dots, C\}$
- Learn a function: $f(\mathbf{x}) : \mathbb{R}^d \mapsto \mathcal{Y}$

Examples of Prediction Problem

- Image Classification:



Which images have birds,
which one does not?

Examples of Prediction Problem

- Image Classification:



Which images have birds,
which one does not?

- Input features $\mathbf{X}$
 - Color histogram
 - $\{(\text{red}, 1004), (\text{blue}, 23000), \dots\}$
- Class label y
 - ‘bird image’: $y = +1$
 - ‘non-bird image’: $y = -1$

Examples of Prediction Problem

- Text categorization:

Doc: Months of campaigning and weeks of round-the-clock efforts in Iowa all came down to a final push Sunday, ...

→ **Topic:**

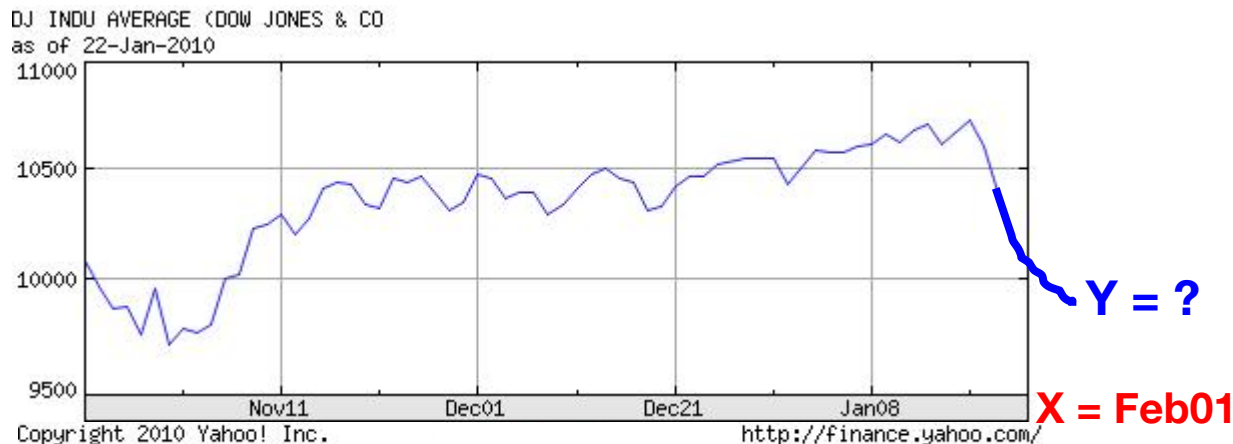
{
Politics
Sport

- Input features $\mathbf{X}$
 - Word frequency
 - $\{(\text{campaigning}, 1), (\text{democrats}, 2), (\text{basketball}, 0), \dots\}$
- Class label:
 - 'Politics': $y = +1$
 - 'Sport': $y = -1$

Examples of Prediction Problem

- When class label is not discrete

Stock Market
Prediction



$$f(\mathbf{x}) : \mathbb{R}^d \mapsto \mathbb{R}$$

Classification

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training set



Learn classifier



Model



Predicted classes

Refund	Marital Status	Taxable Income	Cheat
No	Single	75K	?
Yes	Married	50K	?
No	Married	150K	?
Yes	Divorced	90K	?
No	Single	40K	?
No	Married	80K	?

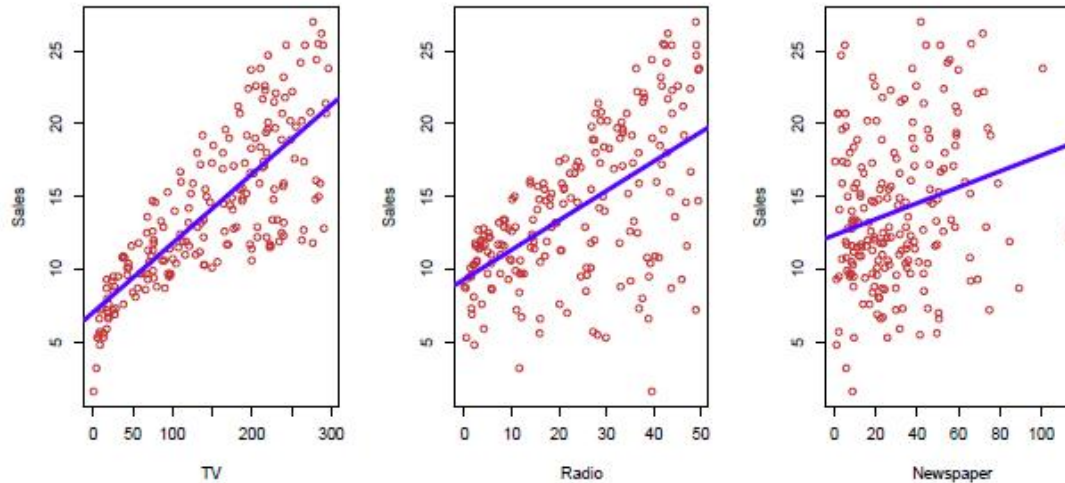
Test set



- Classification and Regression
- Linear Regression
 - OLS/MLE /MAP Estimator
 - Overfitting issue
- Ridge Regression
- Lasso Regression

Regression: Example

- Sales v.s. TV, Radio and Newspaper



- How to predict the sale?
- Design a model:

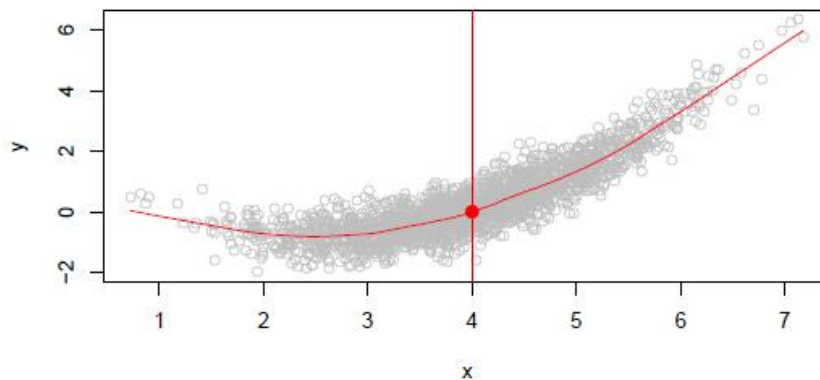
$$\text{Sales} \approx f(\text{TV}, \text{Radio}, \text{Newspaper})$$

Regression Model

- Output $Y \in R$ (e.g., Sales)
- Inputs: $X \in R^p$
 - $X = \begin{pmatrix} X_1 \\ X_2 \\ X_3 \end{pmatrix}$: X_1 : feature TV, X_2 : Radio, X_3 : Newspaper
- Model: $Y = f(X) + \epsilon$
 - f (true regression function): $R^p \rightarrow R$
 - $\epsilon \sim N(0, \sigma^2)$
 - $E(Y|X = x) = f(x)$
- Collecting training data $(X^{(1)}, Y^{(1)}), \dots, (X^{(n)}, Y^{(n)})$ from the model

True regression function

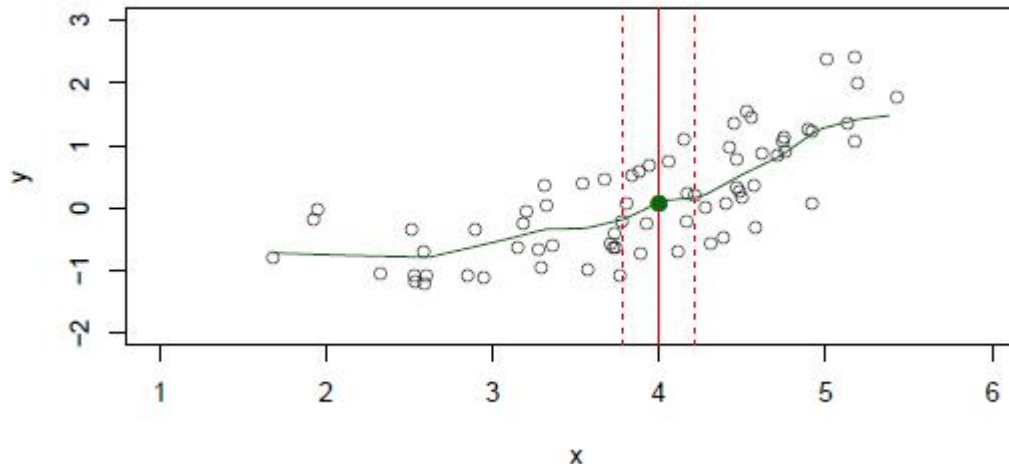
- Model: $Y = f(X) + \epsilon$



- How to predict $X = x$?
- The ideal case: using $f(x) = E(Y|X = x)$
 $f(4) = E(Y|X = 4)$: expected value of Y given $X = 4$
- However, never know $f(x)$, needs an approximation

Local Averaging

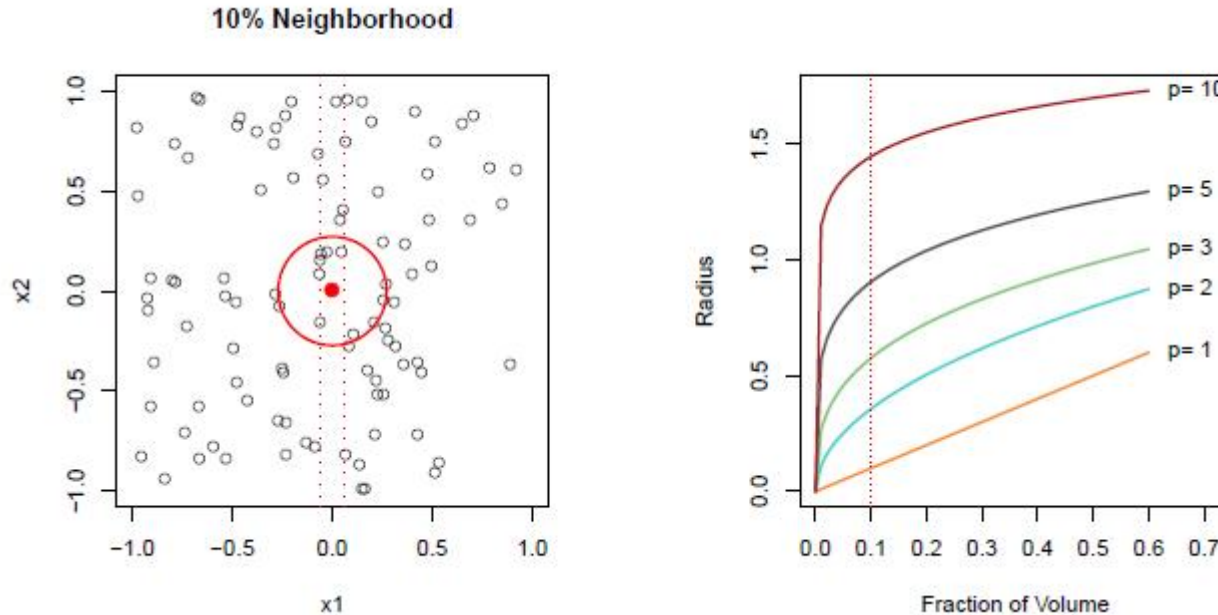
- Cannot compute $E(Y|X = x)$
- Idea: Local Averaging
- Approximate via a neighborhood:
 $\hat{f}(x) = \text{Avg}(Y|X \in N(x))$: $N(x)$ neighborhood of x



Curse of dimensionality

- For small $p \leq 4$, large training data n works well
- When p is large, nearest neighbors tend to be far away in high-dimensions
- Example, we need 10% of training data around x to average y
- When p is large, these data becomes far from x , losing the spirit of local averaging

Curse of dimensionality



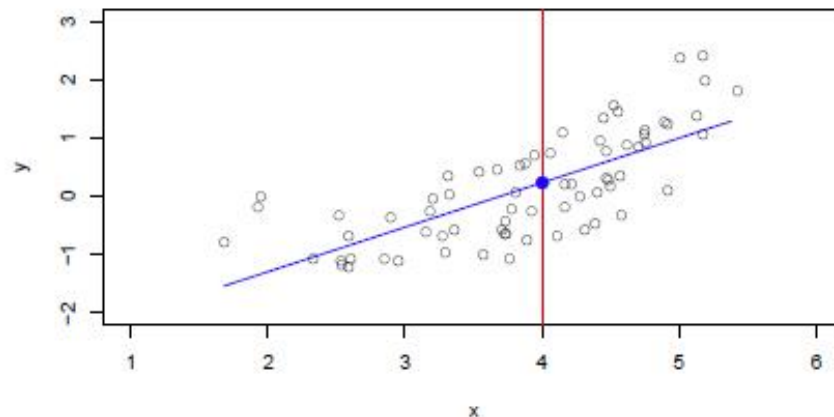
- **Solution 1:** simple model/simple assumption of the form of regression function $f(x) = E(Y|X = x)$
- **Solution 2:** weighted averaging $\Rightarrow$ kernel regression

Linear Model

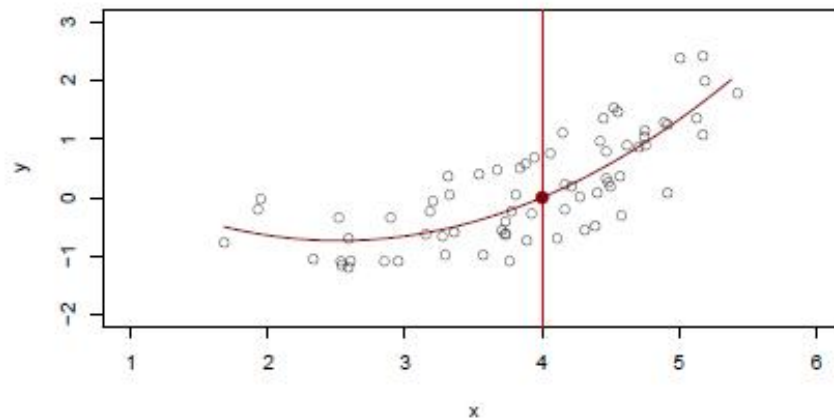
- Linear model:
 - $f_L(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$
 - $p + 1$ parameters: $\beta_0, \beta_1, \dots, \beta_p$
 - Although it could be incorrect, a linear model often serves as a good and interpretable approximation to the unknown true function $f(X)$

Examples

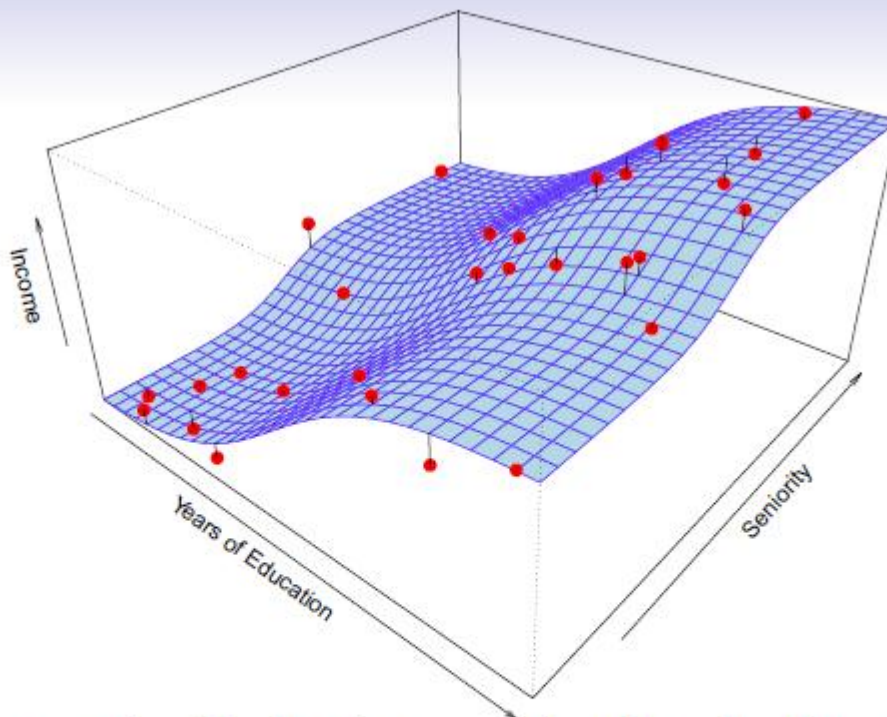
A linear model $\hat{f}_L(X) = \hat{\beta}_0 + \hat{\beta}_1 X$ gives a reasonable fit here



A quadratic model $\hat{f}_Q(X) = \hat{\beta}_0 + \hat{\beta}_1 X + \hat{\beta}_2 X^2$ fits slightly better.



Examples

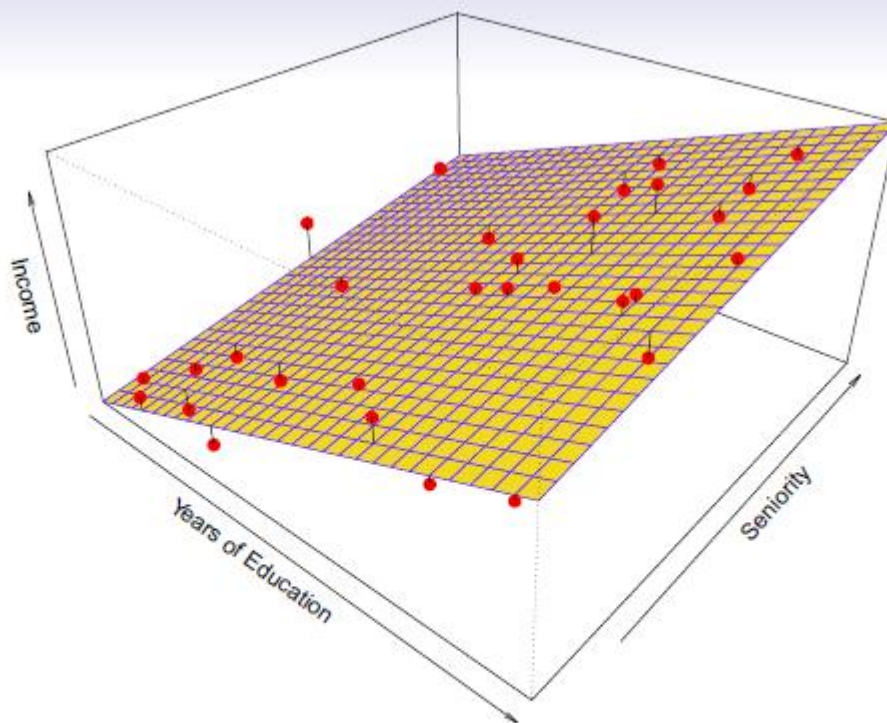


Simulated example. Red points are simulated values for **income** from the model

$$\text{income} = f(\text{education}, \text{seniority}) + \epsilon$$

f is the blue surface.

Examples

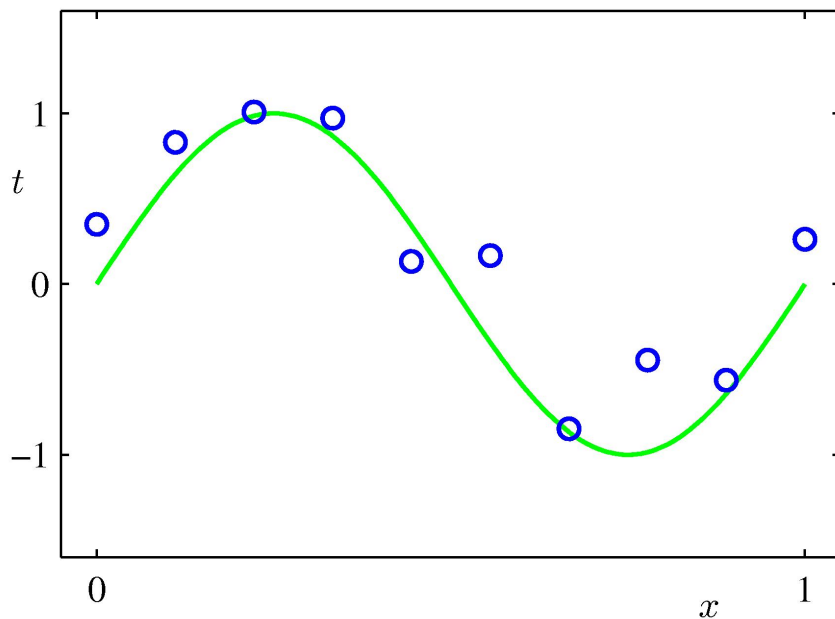


Linear regression model fit to the simulated data.

$$\hat{f}_L(\text{education}, \text{seniority}) = \hat{\beta}_0 + \hat{\beta}_1 \times \text{education} + \hat{\beta}_2 \times \text{seniority}$$

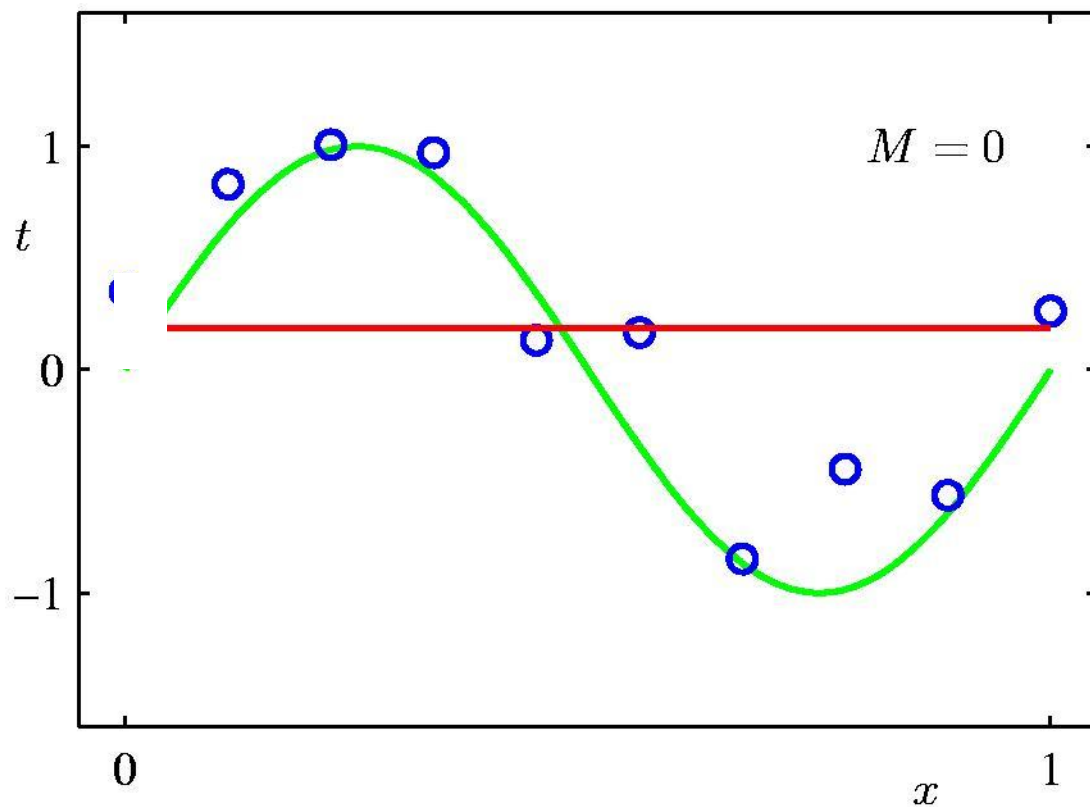
Should We Use a More Flexible Model

Example: Polynomial Curve Fitting

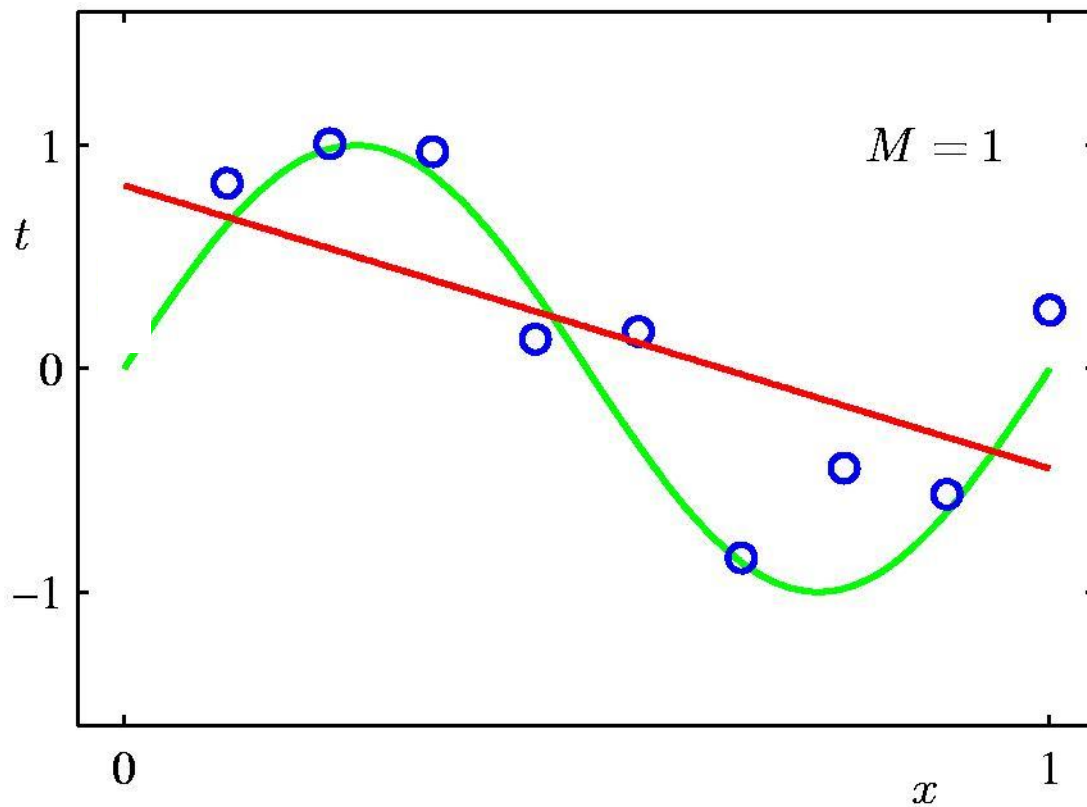


$$\hat{f}(x) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

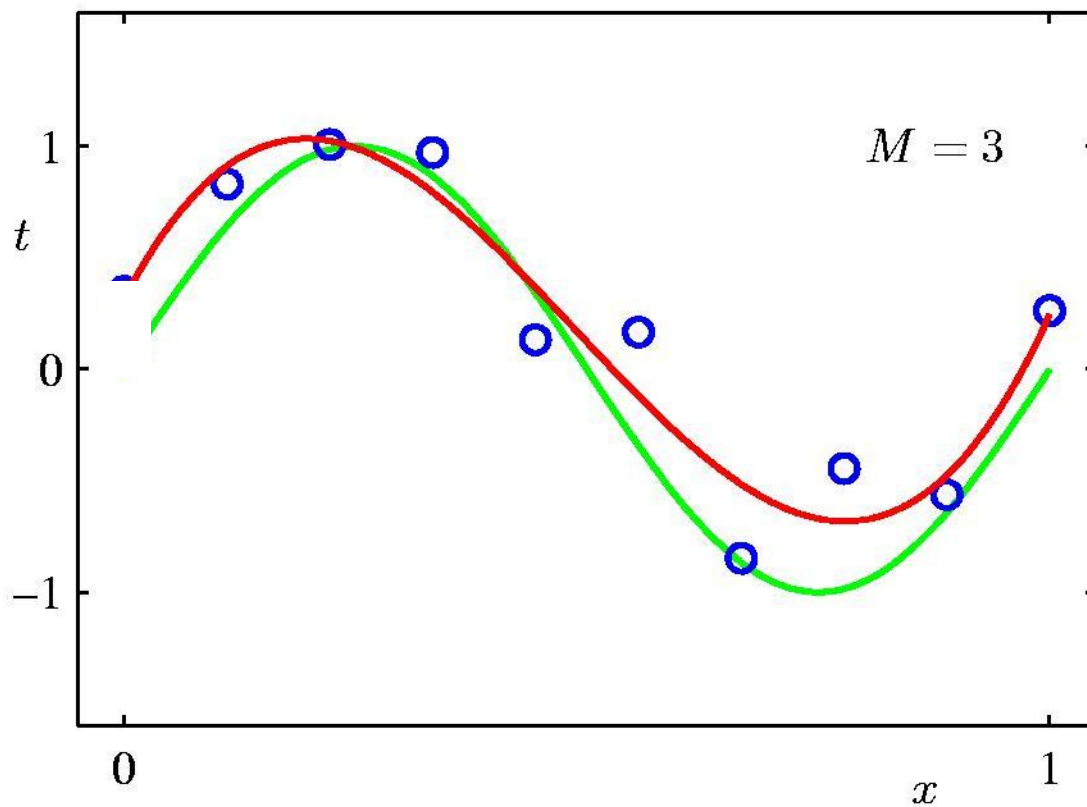
0th Order Polynomial



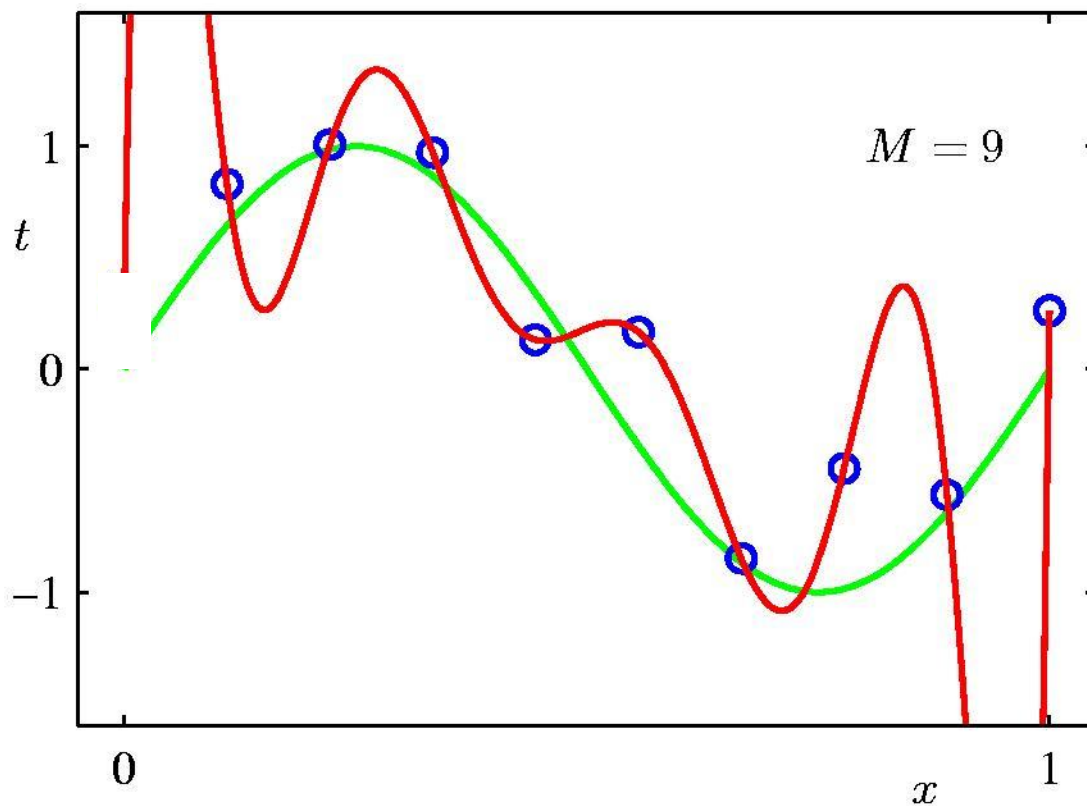
1st Order Polynomial



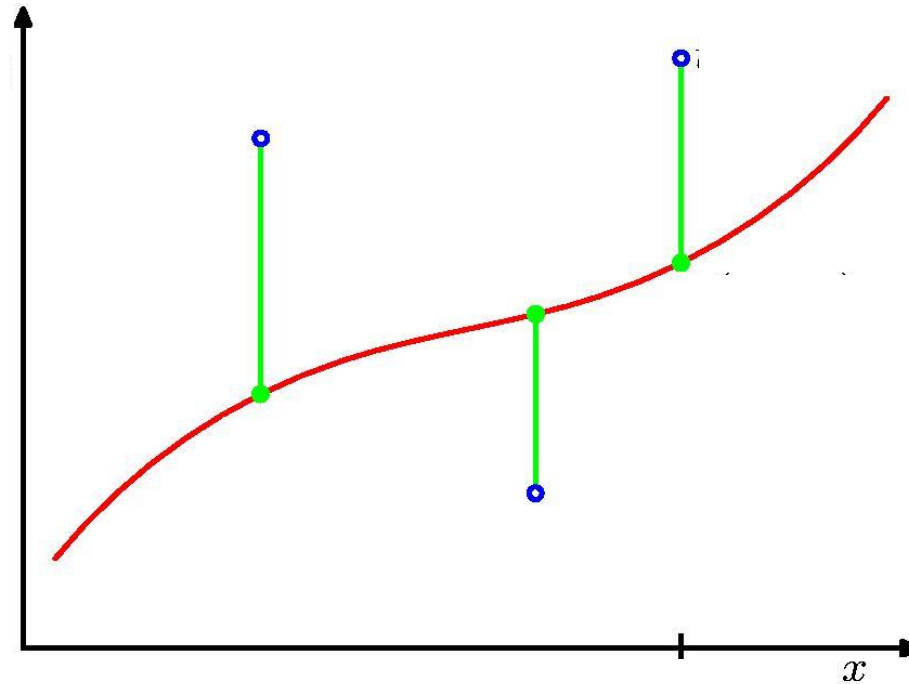
3rd Order Polynomial



9th Order Polynomial

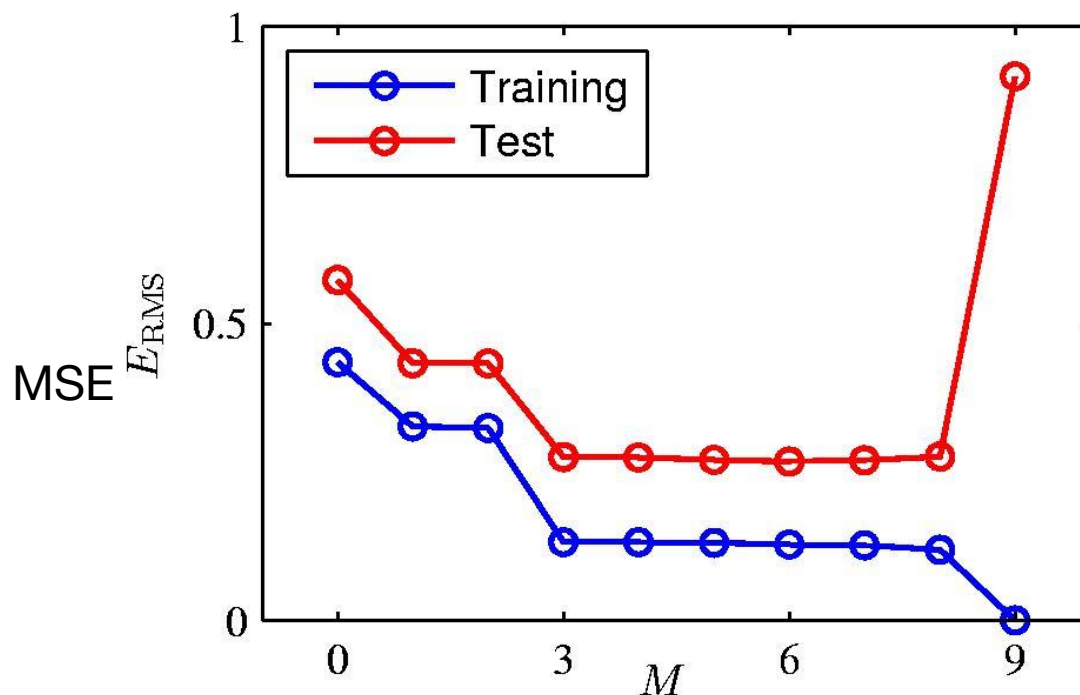


Fitting Criteria: Mean-Squared Error



$$\text{MSE}_{\text{Tr}} = \frac{1}{n} \sum_{i=1}^n \left\{ Y^{(i)} - \hat{f}(X^{(i)}) \right\}^2$$

Over-fitting



Training MSE

$$\text{MSE}_{\text{Tr}} = \frac{1}{n} \sum_{i=1}^n \left\{ Y^{(i)} - \hat{f}(X^{(i)}) \right\}^2$$

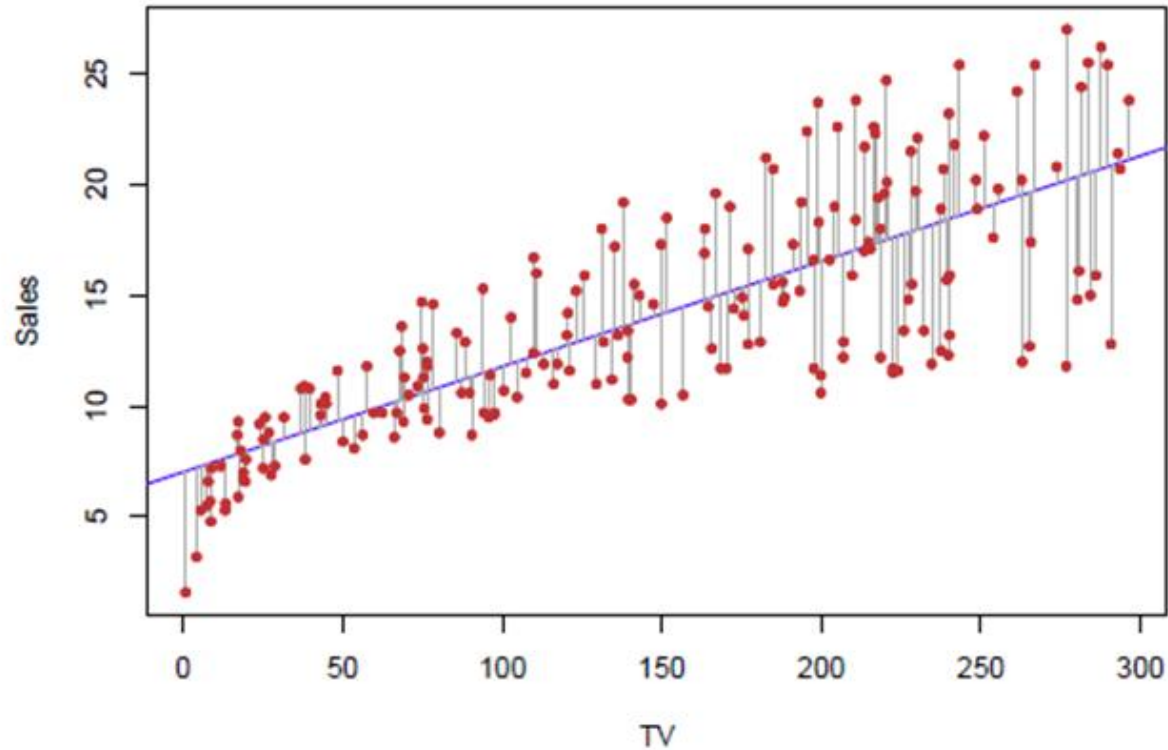
Testing MSE

$$\text{MSE}_{\text{Te}} = \frac{1}{n'} \sum_{i'=1}^{n'} \left\{ Y^{(i')} - \hat{f}(X^{(i')}) \right\}^2$$

Linear Regression with One Predictor

- Model: $Y = \beta_0 + \beta_1 X + \epsilon$
 - β_0 (unknown intercept): average of Y when $X = 0$
 - β_1 (unknown slope): average effect (increase or decrease) in the value Y of one unit increasing of X
- Given training data $(X^{(i)}, Y^{(i)})$, fit the model by minimizing the MSE (mean squared error)
- Least Square Fit: $\min \frac{1}{n} \sum_i (Y^{(i)} - \hat{\beta}_0 - \hat{\beta}_1 X^{(i)})^2$
- Predicted value $\hat{Y}^{(i)} = \hat{\beta}_0 + \hat{\beta}_1 X^{(i)}$

Example



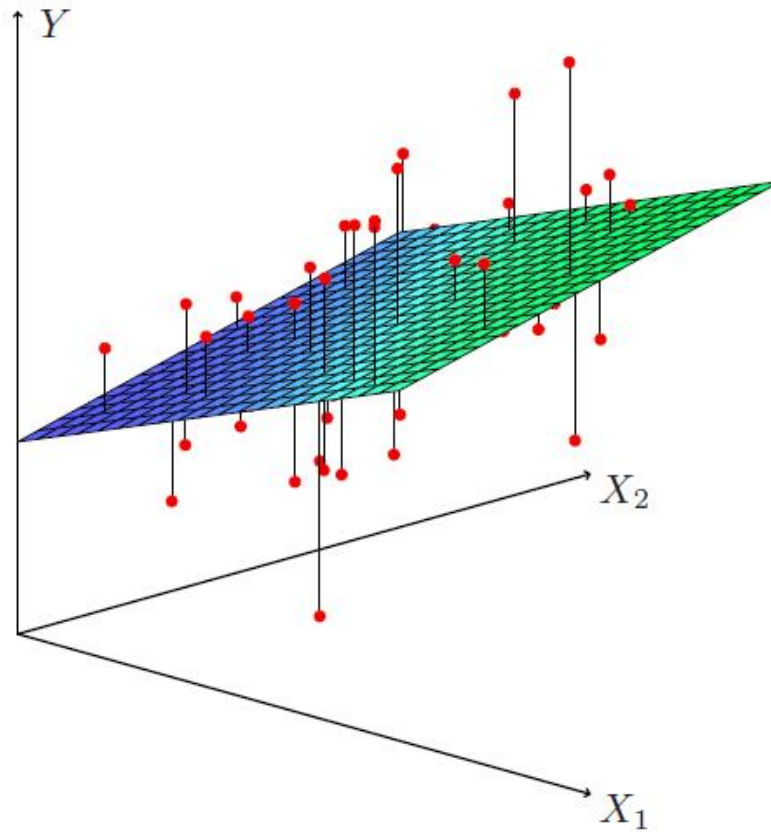
The least squares fit for the regression of **sales** onto **TV**.

Multiple Linear Regression

- Multiple Linear Regression:
 - $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$
 - β_0 : average of Y when all features are zero
 - β_j : average effect (increase of decrease) in the value Y of one unit increasing of X_j when holding all other features fixed
- Sales Example:
$$\text{sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times \text{newspaper} + \epsilon.$$

How to fit the model?

- Minimize the mean squared error



Least Squares Estimator

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n (X^{(i)}\beta - Y^{(i)})^2 \quad X^{(i)} = [1, X_1^{(i)} \dots X_p^{(i)}]$$

$$= \arg \min_{\beta} \frac{1}{n} (\mathbf{A}\beta - \mathbf{Y})^T (\mathbf{A}\beta - \mathbf{Y})$$

$$\mathbf{A} = \begin{bmatrix} X^{(1)} \\ \vdots \\ X^{(n)} \end{bmatrix} = \begin{bmatrix} 1 & X_1^{(1)} & \dots & X_p^{(1)} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & X_1^{(n)} & \dots & X_p^{(n)} \end{bmatrix} \quad \mathbf{Y} = \begin{bmatrix} Y^{(1)} \\ \vdots \\ Y^{(n)} \end{bmatrix}$$

Learned Parameters	$\hat{\beta} = [\hat{\beta}_0, \hat{\beta}_1 \dots \hat{\beta}_p]^T$
Predicted Training Label	$\hat{Y}^{(i)} = X^{(i)}\hat{\beta}$
Predicted Testing Label for new data X	$\hat{f}^L(X) = X\hat{\beta}$

Least Squares Estimator

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{n} (\mathbf{A}\beta - \mathbf{Y})^T (\mathbf{A}\beta - \mathbf{Y}) = \arg \min_{\beta} J(\beta)$$

$$J(\beta) = (\mathbf{A}\beta - \mathbf{Y})^T (\mathbf{A}\beta - \mathbf{Y})$$

$$\left. \frac{\partial J(\beta)}{\partial \beta} \right|_{\hat{\beta}} = 0$$

$$\Rightarrow (\mathbf{A}^T \mathbf{A}) \hat{\beta} = \mathbf{A}^T \mathbf{Y}$$

$$\frac{\partial \beta^T \mathbf{x}}{\partial \mathbf{x}} = \beta$$

$$\frac{\partial \mathbf{x}^T \mathbf{x}}{\partial \mathbf{x}} = 2\mathbf{x}$$

$$\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^T) \mathbf{x}$$

Normal Equations

$$(\mathbf{A}^T \mathbf{A}) \hat{\beta} = \mathbf{A}^T \mathbf{Y}$$

$$(p+1) \times (p+1) \quad (p+1) \times 1 \quad (p+1) \times 1$$

If $(\mathbf{A}^T \mathbf{A})$ is invertible,

$$\hat{\beta} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$$

$$\hat{f}^L(X) = X \hat{\beta}$$

When is $(\mathbf{A}^T \mathbf{A})$ invertible ?

Recall: **Full rank matrices are invertible.** What is rank of $(\mathbf{A}^T \mathbf{A})$?

What if $(\mathbf{A}^T \mathbf{A})$ is not invertible ? **Regularization (later)**

Fast Computation

Even when $\mathbf{A}^T \mathbf{A}$ is invertible, might be computationally expensive if $\mathbf{A}$ is huge.

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{n} (\mathbf{A}\beta - \mathbf{Y})^T (\mathbf{A}\beta - \mathbf{Y}) = \arg \min_{\beta} J(\beta)$$

What should we do?

Convex optimization!

Gradient Descent !

Least Square Estimator

$$J(\beta) = (\mathbf{A}\beta - \mathbf{Y})^T (\mathbf{A}\beta - \mathbf{Y})$$



$$\nabla^2 J(\beta) = 2\mathbf{A}^T \mathbf{A} \succeq 0$$



$J(\beta)$ is a convex function



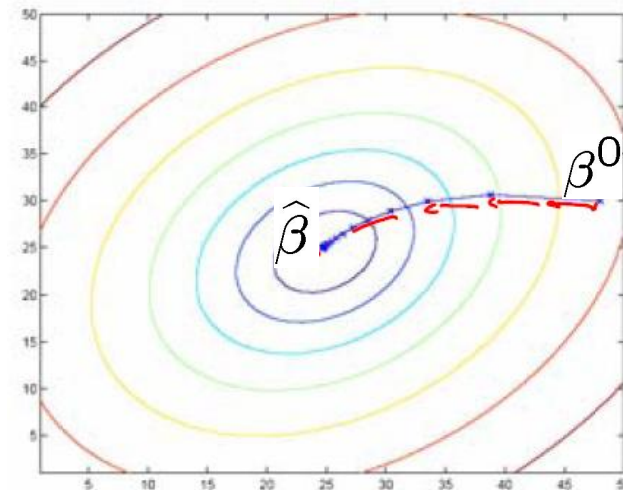
We are able to find the optimal parameters

Gradient Descent

Gradient Descent since $J(\beta)$ is convex

Initialize: β^0

$$\begin{aligned} \text{Update: } \beta^{t+1} &= \beta^t - \alpha_t \left. \frac{\partial J(\beta)}{\partial \beta} \right|_t \\ &= \beta^t - 2\alpha_t \underbrace{\mathbf{A}^T (\mathbf{A}\beta^t - \mathbf{Y})}_{0 \text{ if } \beta^t = \hat{\beta}} \end{aligned}$$

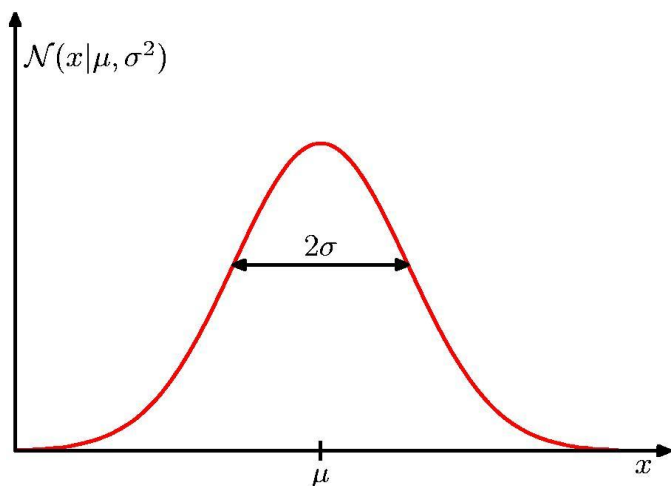


Stop: when some criterion met e.g. fixed # iterations $\left| \left. \frac{\partial J(\beta)}{\partial \beta} \right|_{\beta^t} \right| \leq \gamma < \epsilon$.

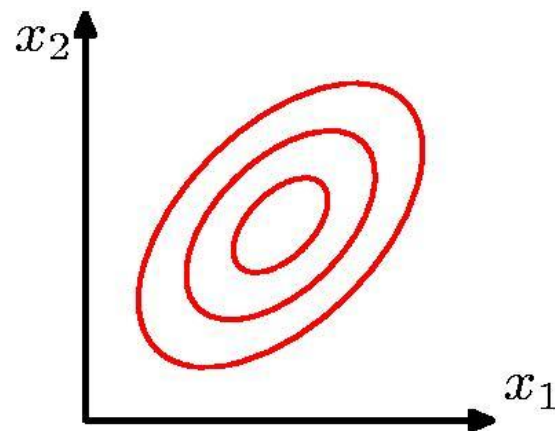
Linear Regression: MLE

Multiple Linear Model

$$Y = \mathbf{A}\beta + \epsilon \quad \epsilon \sim \mathcal{N}(0, \sigma^2 \mathbf{I}) \quad \longrightarrow \quad Y \sim \mathcal{N}(\mathbf{A}\beta, \sigma^2 \mathbf{I})$$



$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp \left\{ -\frac{1}{2\sigma^2} (x - \mu)^2 \right\}$$



$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\boldsymbol{\Sigma}|^{1/2}} \exp \left\{ -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right\}$$

MLE: Gaussian Distribution

$$Y \sim \mathcal{N}(\mathbf{A}\beta, \sigma^2 \mathbf{I}) \quad \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\boldsymbol{\Sigma}|^{1/2}} \exp \left\{ -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) \right\}$$



$$p(Y|\mathbf{A}\beta, \sigma^2 \mathbf{I}) = \frac{1}{(\sqrt{2\pi}\sigma)^n} \exp \left\{ -\frac{1}{2\sigma^2} (Y - \mathbf{A}\beta)^T (Y - \mathbf{A}\beta) \right\}$$



$$\log p(Y|\mathbf{A}\beta, \sigma^2 \mathbf{I}) = -\frac{1}{2\sigma^2} (Y - \mathbf{A}\beta)^T (Y - \mathbf{A}\beta) - \text{const}$$

MLE: Gaussian Distribution

$$\log p(Y|\mathbf{A}\beta, \sigma^2\mathbf{I}) = -\frac{1}{2\sigma^2}(Y - \mathbf{A}\beta)^T(Y - \mathbf{A}\beta) - \text{const}$$



$$\hat{\beta}_{\text{MLE}} = \arg \max_{\beta} \underbrace{\log p(\{(X^{(i)}, Y^{(i)})\}_{i=1}^n | \beta, \sigma^2)}_{\text{log likelihood}}$$

$$= \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - X^{(i)}\beta)^2 = \hat{\beta}$$



**Least Square Estimate is same as
Maximum Likelihood Estimate under a
Gaussian model !**

Case Study: Linear Regression

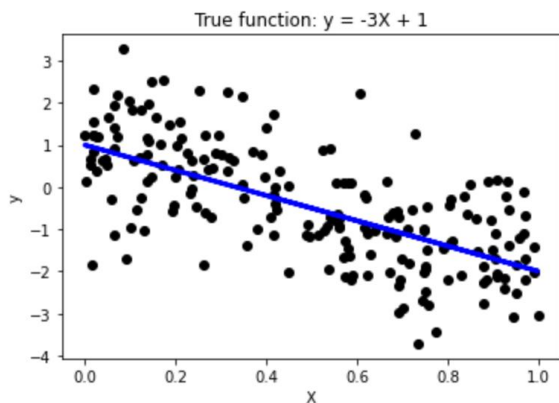
- **Synthetic Data**

```
In [1]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt

seed = 1           # seed for random number generation
numInstances = 200 # number of data instances
np.random.seed(seed)
X = np.random.rand(numInstances,1).reshape(-1,1)
y_true = -3*X + 1
y = y_true + np.random.normal(size=numInstances).reshape(-1,1)

plt.scatter(X, y, color='black')
plt.plot(X, y_true, color='blue', linewidth=3)
plt.title('True function: y = -3X + 1')
plt.xlabel('X')
plt.ylabel('y')
```

Out[1]: Text(0, 0.5, 'y')



Get Training and Testing Data

Step 1: Split Input Data into Training and Test Sets

```
In [2]: ▶ numTrain = 20    # number of training instances
        numTest = numInstances - numTrain

        X_train = X[:-numTest]
        X_test = X[-numTest:]
        y_train = y[:-numTest]
        y_test = y[-numTest:]
```

Step 2: Fit Regression Model to Training Set

```
In [3]: ▶ from sklearn import linear_model
        from sklearn.metrics import mean_squared_error, r2_score

        # Create linear regression object
        regr = linear_model.LinearRegression()

        # Fit regression model to the training set
        regr.fit(X_train, y_train)
```

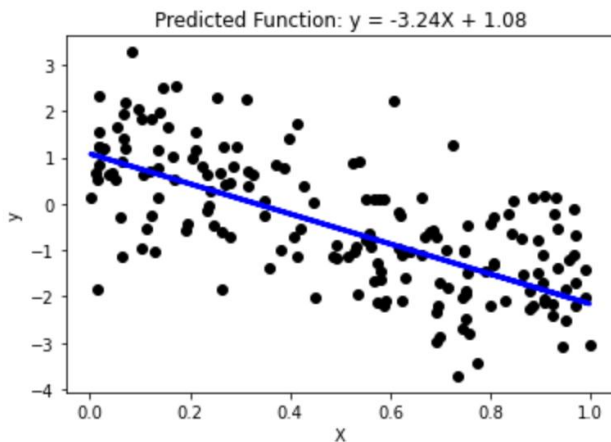
Model Fit

```
In [6]: ► # Display model parameters
print(' Slope = ', regr.coef_[0][0])
print(' Intercept = ', regr.intercept_[0])### Step 4: Postprocessing

# Plot outputs
plt.scatter(X_test, y_test, color='black')
plt.plot(X_test, y_pred_test, color='blue', linewidth=3)
titlestr = 'Predicted Function: y = %.2fX + %.2f' % (regr.coef_[0], regr.intercept_[0])
plt.title(titlestr)
plt.xlabel('X')
plt.ylabel('y')
```

Slope = -3.242354544656501
Intercept = 1.0805993038584834

Out[6]: Text(0, 0.5, 'y')





Exercise: Boston Housing

- Load data MASS
 - `library(MASS)`
 - `attach(Boston)`
 - See the data by the command: `?Boston` (e.g., `mdev`: median value of owner-occupied homes
`lstat`: lower status of the population)
- Conduct linear regression (using command `lm`)
 - `medv ~ lstat`
 - `medv ~ lstat+age`
 - `medv~.` (on all the variables)
 - `medv~.-age` (all the variables except for age)
 - `lm(mdev~lstat*age,data=Boston)` (interaction terms between `lstat` * `age`)
 - `medv~lstat+lstat^2` (`lstat` and `lstat^2`)
- Conduct linear regression
 - Output summary fitted linear regression (using command `summary`)

Exercise: Boston Housing

This data frame contains the following columns:

- Crim: per capita crime rate by town.
- Zn: proportion of residential land zoned for lots over 25,000 sq.ft.
- Indus: proportion of non-retail business acres per town.
- Chas: Charles River dummy variable (= 1 if tract bounds river; 0 otherwise).
- Nox: nitrogen oxides concentration (parts per 10 million).
- Rm: average number of rooms per dwelling.
- Age: proportion of owner-occupied units built prior to 1940.
- Dis: weighted mean of distances to five Boston employment centres.
- Rad: index of accessibility to radial highways.
- Tax: full-value property-tax rate per $\$10,000$.
- Ptratio: pupil-teacher ratio by town.
- Black: $1000(Bk - 0.63)^2$ where Bk is the proportion of blacks by town.
- Lstat: lower status of the population (percent).
- Medv: median value of owner-occupied homes in $\$1000$ s.



Exercise: Boston Housing

- Load data
- Conduct linear regression
 - $\text{medv} \sim \text{lstat}$
 - $\text{medv} \sim \text{lstat} + \text{age}$
 - $\text{Medv} \sim \text{all the variables}$
 - $\text{medv} \sim \text{lstat} * \text{age}$
 - $\text{medv} \sim \text{lstat} + \text{lstat}^2$
- Conduct linear regression
 - Output summary fitted linear regression



Outline

- Classification and Regression
- Linear Regression
 - OLS/MLE /MAP Estimator
 - Overfitting issue
- Ridge Regression
- Lasso Regression

Linear Regression: MAP

- In least square estimator, we have

$$\hat{\beta} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$$

- What if $(\mathbf{A}^T \mathbf{A})$ is not invertible ?
- We are going to try MAP estimator

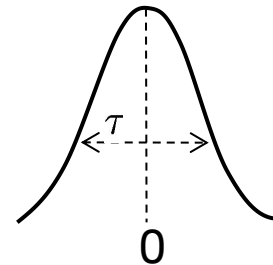
$$\hat{\beta}_{\text{MAP}} = \arg \max_{\beta} \underbrace{\log p(\{(X^{(i)}, Y^{(i)})\}_{i=1}^n | \beta, \sigma^2})}_{\text{log likelihood}} + \underbrace{\log p(\beta)}_{\text{log prior}}$$

Ridge Regression (岭回归)

Ridge Regression: Gaussian Prior

$$\beta \sim \mathcal{N}(0, \tau^2 \mathbf{I})$$

$$p(\beta) \propto e^{-\beta^T \beta / 2\tau^2}$$



$$\hat{\beta}_{\text{MAP}} = \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - X^{(i)} \beta)^2 + \lambda \|\beta\|_2^2$$

$\downarrow$
constant(σ^2, τ^2)

Ridge Regression

$$\hat{\beta}_{\text{MAP}} = (\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^T \mathbf{Y}$$

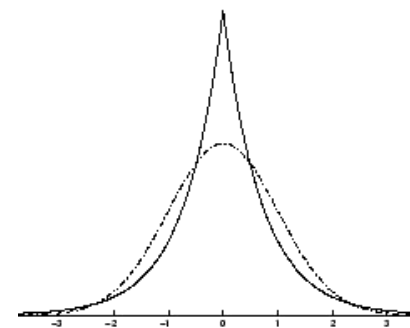
➡ $(\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})$ Invertible when $\lambda > 0$

Lasso Regression

Lasso Regression: Laplace Prior

$$\beta_j \stackrel{iid}{\sim} \text{Laplace}(0, t)$$

$$p(\beta_j) \propto e^{-|\beta_j|/t}$$



$$\hat{\beta}_{\text{MAP}} = \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - X^{(i)}\beta)^2 + \lambda \|\beta\|_1$$

$\downarrow$
constant(σ^2, t)

Feature Selection and Regression

- R^2
- Ridge/Lasso Regression

$$\hat{\beta}_{\text{MAP}} = \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - X^{(i)}\beta)^2 + \lambda \|\beta\|_2^2 \quad \text{Ridge Regression}$$

$\downarrow$
constant(σ^2, τ^2)

$$\hat{\beta}_{\text{MAP}} = \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - X^{(i)}\beta)^2 + \lambda \|\beta\|_1$$

$\downarrow$
constant(σ^2, t)

Lasso Regression

- Elastic Net Regularization

Example

- *# data from textir package*
- **data**(we8there)
- **dim**(we8thereCounts)

- Data are very high dimensional.
- But fortunately not many words per document (features are sparse)

Sample Examples

5.4 Ridge Regression

Ridge regression is a variant of MLR designed to fit a linear model to the dataset by minimizing the following regularized least-square loss function:

$$L_{\text{ridge}}(y, f(X, w)) = \sum_{i=1}^N \|y_i - X_i w - w_0\|^2 + \alpha \left[\|w\|^2 + w_0^2 \right],$$

where α is the hyperparameter for ridge regression. Note that the ridge regression model reduces to MLR when $\alpha = 0$. By increasing the value of α , we can control the complexity of the model as will be shown in the example below.

In the example shown below, we fit a ridge regression model to the previously created training set with correlated attributes. We compare the results of the ridge regression model against those obtained using MLR.

```
from sklearn import linear_model

ridge = linear_model.Ridge(alpha=0.4)
ridge.fit(X_train5, y_train)
y_pred_train_ridge = ridge.predict(X_train5)
y_pred_test_ridge = ridge.predict(X_test5)

model6 = "%.2f X + %.2f X2 + %.2f X3 + %.2f X4 + %.2f X5 + %.2f" % (ridge.coef_[0][0],
                                                                    ridge.coef_[0][1], ridge.coef_[0][2],
                                                                    ridge.coef_[0][3], ridge.coef_[0][4], ridge.intercept_[0])
values6 = [ model6, np.sqrt(mean_squared_error(y_train, y_pred_train_ridge)),
            np.sqrt(mean_squared_error(y_test, y_pred_test_ridge)),
            np.absolute(ridge.coef_[0]).sum() + np.absolute(ridge.intercept_[0])]

ridge_results = pd.DataFrame([values6], columns=columns, index=['Ridge'])
pd.concat([results, ridge_results])
```

5.5 Lasso Regression

One of the limitations of ridge regression is that, although it was able to reduce the regression coefficients associated with the correlated attributes and reduce the effect of model overfitting, the resulting model is still not sparse. Another variation of MLR, called lasso regression, is designed to produce sparser models by imposing an ℓ_1 regularization on the regression coefficients as shown below:

$$L_{\text{lasso}}(y, f(X, w)) = \sum_{i=1}^N \|y_i - X_i w - w_0\|^2 + \alpha \left[\|w\|_1 + |w_0| \right]$$

The example code below shows the results of applying lasso regression to the previously used correlated dataset.

```
❏ from sklearn import linear_model

lasso = linear_model.Lasso(alpha=0.02)
lasso.fit(X_train5, y_train)
y_pred_train_lasso = lasso.predict(X_train5)
y_pred_test_lasso = lasso.predict(X_test5)

model7 = "%.2f X + %.2f X2 + %.2f X3 + %.2f X4 + %.2f X5 + %.2f" % (lasso.coef_[0],
                                                                    lasso.coef_[1], lasso.coef_[2],
                                                                    lasso.coef_[3], lasso.coef_[4], lasso.intercept_[0])
values7 = [ model7, np.sqrt(mean_squared_error(y_train, y_pred_train_lasso)),
            np.sqrt(mean_squared_error(y_test, y_pred_test_lasso)),
            np.absolute(lasso.coef_[0]).sum() + np.absolute(lasso.intercept_[0])]

lasso_results = pd.DataFrame([values7], columns=columns, index=['Lasso'])
pd.concat([results, ridge_results, lasso_results])
```

Exercise: Boston Housing

This data frame contains the following columns:

- Crim: per capita crime rate by town.
- Zn: proportion of residential land zoned for lots over 25,000 sq.ft.
- Indus: proportion of non-retail business acres per town.
- Chas: Charles River dummy variable (= 1 if tract bounds river; 0 otherwise).
- Nox: nitrogen oxides concentration (parts per 10 million).
- Rm: average number of rooms per dwelling.
- Age: proportion of owner-occupied units built prior to 1940.
- Dis: weighted mean of distances to five Boston employment centres.
- Rad: index of accessibility to radial highways.
- Tax: full-value property-tax rate per \ \$10,000.
- Ptratio: pupil-teacher ratio by town.
- Black: $1000(Bk - 0.63)^2$ where Bk is the proportion of blacks by town.
- Lstat: lower status of the population (percent).
- Medv: median value of owner-occupied homes in \ \$1000s.



Other Regression Models

- ✓ Polynomial regression
- ✓ Regression with nonlinear features
- ✓ Kernel regression - Local/Weighted regression
- ✓ Regression trees



Some Takeaways

- Linear Regression: OLS/MLE/MAP
- Ridge Regression
- Lasso Regression

The End

■ THANK YOU

Any questions?

